1. List out the lexical tokens in verilog and explain them

A verilog source code is made up of different types of tokens.

The main types of lexical tokens are

1. Keywords
2. Identifiers
3. Numbers
4. strings
5. Operators
6. special characters / separators
7. comments

1. Keywords:

Keywords are reserved words in verilog that have a predefined meaning. They cannot normally be used as identifiers.

ex:

```
module adder;
input a, b;
output
Output sum;
end module.
```

Scanned with OKEN Scanner

2. Identifiers:

Identifiers are the names given to modules, signals, variables etc.

ex:

```
module counter;
 reg clk;
 wire output;
```

3. Numbers:

Numbers represent constant values in a verilog program they can be written in decimal, binary, octal, or hexadecimal formats.

ex:

8'b10101010

8'd 25

8'o 17

8 h AF

4. Strings:

Strings are sequences of characters enclosed in double Quotation marks (" ")

Ex:

$display ("Hello Verilog").

5. Operators:

Operators are symbols used to perform operations on values.

Scanned with OKEN Scanner

ex:

Arthemetic +, -, *, /

Relational >, <, ==

logical &&, ||, !

Bitwise &, |, ^, ~.

ex:

assign sum = a+b;

6. Special characters / separators:

These symbols separate different parts of verilog program.

Ex: input [3:0] a, b.

7. Comments

Comments are used to describe or explain the code and are ignored by the verilog compiler.

Single line comment

// This is a comment

multi-line comment

/*

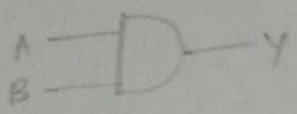
*/

2. Explain module structure with the help of example.

A module is the basic building block of a verilog HDL program, It describes the inputs, outputs, and internal operation of a digital circuit.

Scanned with OKEN Scanner

ex: 8input AND gate




```
module and-gate (
    input a,
    input b,
    output y,
);
  assign y=a&b;
endmodule
```

explanation:

module operation

module and_gate starts the module and gives it the name and-gate.

- port list:

a, b, & y are the ports of the module

a & b → input ports

y → output ports

→ Port Declaration

the input & output keywords specify the direction of signals.

→ Logic Description

assign y=a&b;

this implementation the AND operation

→ end module indicate the end of the module.

Scanned with OKEN Scanner

3. Describe simulation and synthesis tool.

In verilog HDL, simulation and synthesis are two important processes used to design and verify digital circuits.

1. Simulation

Simulation the proces of checking the behaviour & functionality of a verilog design before implementing it in hardware. A simulation tool executes the verilog code and produces waveforms or output values for different input conditions.

ex of simulation tools:

- Modelsim
- Questa
- Icarus verilog
- xsim

ex:

for An AND gate

verilog:

assign y=a & b;

the simulator checks whether the output y becomes 1 only when both a & b are 1

Simulation flow

Verilog Design + Test bench

↓

simulation tool

↓

Waveform / Results

↓

Verify functionally

2. Synthesis:

Synthesis is the process of converting a verilog HDL description into a hardware representation such as Logic gates, flip flops, and other digital components

A synthesis tool ~~com~~ analyzes the verilog code & generates a netlist that can be implemented on a FPGA or ASIC

ex. of synthesis tools: → xilinx vivado, Intel Quartus, Synopsys Design Compiler.

Synthesis flow:

Verilog RTL code → synthesis tool →

logic Netlist → FPGA / ASIC Implementation.